Vibe Coding prompt

Act as a Solidity and Python Web3 developer. Build map buyer requirements to ether escrow for an isolated test chain. Use Solidity 0.8.30 and web3.py 8.0.0 with EthereumTesterProvider backed by Py-EVM; also allow --rpc for an isolated unlocked local JSON-RPC test chain. Requirements: Created → funded → released; exactly 0.1 ETH is paid once and only the buyer authorizes release. Read mock-data.json, derive a canonical SHA-256 digest where required, compile using pinned solc-js 0.8.30, deploy actual EVM bytecode, wait for every receipt and assert receipt status. Include positive and negative authorization/state tests, output chain ID/address/transaction hashes/gas and a deployment manifest. Do not simulate contract state in Python or use mainnet keys. Use OpenZeppelin 5.4.0 for ERC standards. Return Contract.sol, run_activity.py, mock-data.json, compile.cjs, requirements.txt and a detailed troubleshooting guide. Explain integer units, failure cases and what the assertions prove. Never claim a browser wallet or public testnet succeeded unless actually exercised.

Review prompt

Review every external function for authorization, state transitions, integer units and invariants. Point to exact lines. Produce a requirement-to-assertion matrix and add a failing test before repairing any defect.

Scenario-specific generation constraints

Read amount_eth from the mock record and convert using to_wei. Constructor seller=account1, buyer=account0. fund() with price minus1 wei must revert; fund exact price succeeds. Outsider release fails; buyer release increases seller balance by exactly price; second release fails.

Expected assertions: funded=true; released=true; escrow balance zero; seller gains0.1ETH.

Extension prompt

Add a delivery evidence digest requirement before buyer release, and add a failing test before editing Solidity. Return the exact changed source lines and positive/negative tests.